

732A94 Advanced R programming

Lecture 6, part 1: Optimizing code and computational complexity

Johan Alenlöv (Slides based on Leif Jonssons, Måns Magnusson and Krzysztof Bartoszek)

Linköpings Universitet

- Optimizing code
 - Profiling
 - Writing performant code
- Computational complexity
 - Classes of problems
 - Big-Oh notation
 - Determining complexity
 - Sorting
 - The master theorem
 - Approximation algorithms

Questions?

Any questions from last time?

“Programmers waste enormous amounts of time thinking about, or worrying about, the speed of noncritical parts of their programs, and these attempts at efficiency actually have a strong negative impact when debugging and maintenance are considered.”

Depends on many things:

1. Code
2. Complexity
3. Compiler
4. Hardware
5. Language

If you don't measure, you don't optimize!

0. Choose the optimal algorithm
1. Write code that works, with an accompanying test suite
2. Profile your code for bottlenecks
3. Try to eliminate the bottlenecks
4. Redo 2–3 until fast enough

`proc.time()` is a basic starting tool.

Measuring: `system.time()`

```
x <- runif(1e6)
system.time(sort(x))
```

```
##      user  system elapsed
##    0.032    0.003    0.036
```

`elapsed` is wall-clock time, `user` is CPU time in R, `system` is CPU time in the operating system.

For anything faster than a few milliseconds `system.time()` is too crude – use `microbenchmark`.

Measuring: microbenchmark

```
library(microbenchmark)
x <- runif(1000)
microbenchmark(loop = {s <- 0; for (i in x) s <- s + i},
               vec  = sum(x),
               times = 100)
```

```
## Unit: nanoseconds
```

##	expr	min	lq	mean	median	uq	max	neval	cldev
##	loop	404711	432263	479768.88	449770	466436.5	2443805	100	a
##	vec	902	984	1077.48	1025	1107.0	2173	100	b

Measuring: `bench::mark()`

```
library(bench)
x <- runif(1000)
mark(loop = {s <- 0; for (i in x) s <- s + i; s},
      vec = sum(x), iterations = 200)[, 1:5]
```

```
## # A tibble: 2 x 5
```

##	expression	min	median	`itr/sec`	mem_alloc
##	<bch:expr>	<bch:tm>	<bch:tm>	<dbl>	<bch:byt>
## 1	loop	409us	432.92us	2178.	8.6KB
## 2	vec	943ns	1.02us	973073.	0B

`mark()` checks that the alternatives return the same result before it times them, and errors if they do not. It also reports memory allocation – the loop allocates, `sum()` does not.

```
fgen <- function() {  
  rt(10^5, df = 10); rgamma(10^5, shape = 1); rcauchy(10^5)  
}  
fprint <- function() print(1:10^5)  
  
Rprof(tmp <- tempfile(), line.profiling = TRUE,  
      memory.profiling = TRUE, interval = 0.01)  
## interval: time between samples  
for (i in 1:10) { fgen(); fprint() }  
Rprof(NULL)  
summaryRprof(tmp, memory = "both")
```

Example after P. Biecek, *Programowanie w R*, p. 151, 153. See also `RprofEx.R`.

Profiling: the output

\$by.self

	self.time	self.pct	total.time	mem.total
"print.default"	2.37	87.13	2.37	0.1
"rt"	0.14	5.15	0.14	6.9
"rgamma"	0.11	4.04	0.11	7.7
"rcauchy"	0.10	3.68	0.10	7.7

\$by.total

	total.time	total.pct	mem.total	self.time
"print.default"	2.37	87.13	0.1	2.37
"fprintf"	2.37	87.13	0.1	0.00
"fgen"	0.35	12.87	22.2	0.00

The bottleneck is *printing*, not the random number generation – which is not what you would have guessed.

```
profvis::profvis({  
  y <- matrix(rnorm(10^6), 1000, 1000) *  
    matrix(rexp(10^6), 1000, 1000)  
  x <- sort(rnorm(10^6))  
})
```

profvis gives an interactive flame graph in RStudio, line by line.

bench::mark() (previous slide) is the safety net: it will not let you “optimise” your way to a different answer.

0. Optimal data structure and algorithm
1. Look for existing solutions
2. Do less work
3. Vectorise
4. Parallelize
5. Avoid copies

Always in that order. Parallelising a bad algorithm is still a bad algorithm.

```
x <- runif(1e5)
f_loop <- function(x) {
  out <- numeric(length(x))
  for (i in seq_along(x)) out[i] <- x[i]^2
  out
}
system.time(f_loop(x))["elapsed"]
```

```
## elapsed
##    0.004
```

```
system.time(x^2)["elapsed"]
```

```
## elapsed
##         0
```

Avoid copies: preallocate

```
grow <- function(n) {  
  out <- c() # copies every time  
  for (i in 1:n) out <- c(out, i)  
  out  
}  
prealloc <- function(n) {  
  out <- numeric(n)  
  for (i in 1:n) out[i] <- i  
  out  
}
```

Avoid copies: preallocate

```
system.time(grow(2e4))["elapsed"]
```

```
## elapsed
```

```
##    0.262
```

```
system.time(prealloc(2e4))["elapsed"]
```

```
## elapsed
```

```
##    0.002
```

Growing a vector with `c()` re-allocates and copies the whole thing on every iteration – that is $O(n^2)$ work for an $O(n)$ task.

Speed is important! (do not forget memory)

Time to write code

Time to maintain (understand) code

Time to execute code

"You can have it Good, Fast, Cheap.
Pick any two."

1. Performance
2. Complexity

Complexity affects performance.

Performance is what you measure on *your* machine with *this* input. Complexity is what happens as the input grows.

Theoretical worst case (but what about the average case?)

- Big-Oh notation
- Basic operations
- Relationship between number of operations and problem size

- Time complexity
- Space (memory) complexity
- Worst case complexity
- Average case complexity

(Theoretical) data structures

- Matrix (data frame, list)
- List (NOT in the R sense, but with pointers), FIFO, LIFO
- Sets (no particular order of elements, cannot index)
- Graphs (vertex, edge): vertex adjacency matrix, vertex adjacency list

The data structure decides which operations are cheap – that is where complexity is won or lost.

Decision problems the answer is yes or no, e.g. is x a prime number?

Optimization problems find an object satisfying a certain property, e.g. the largest prime smaller than $x + 1$

Non-algorithmic problems cannot be solved by an algorithm, e.g. the halting problem: does a given algorithm end in finite time or fall into an infinite loop?

Presumably non-algorithmic problems

No algorithm is known, but we do not know that none exists – e.g. the Collatz problem:

```
repeat {  
  if (k %% 2 == 0) { k <- k / 2 } else { k <- 3 * k + 1 }  
  if (k == 1) break  
}
```

Does it halt for every k ?

(Still actively researched, e.g. [arXiv:2106.11859](https://arxiv.org/abs/2106.11859))

Polynomial problems can be solved by an algorithm whose running time is bounded by a polynomial in the input size, e.g. sorting n elements.

Non-polynomial problems cannot, e.g. generating all permutations of an n element set, $n!$.

P the class of polynomial problems.

NP nondeterministic polynomial: there exists a polynomial time procedure that *verifies* whether something is an admissible solution, e.g. checking whether a graph colouring is admissible.

$$P \subset NP \text{ but } P \stackrel{???}{=} NP$$

NP-complete every problem in NP can be reduced to it in polynomial time, e.g. bin packing, **knapsack**, longest common subsequence, chromatic number of a graph, TSP, satisfiability (SAT).

NP-hard if it can be solved in polynomial time then $\text{SAT} \in P$.

“How fast does a function grow?”

$$f(n) = O(g(n)) \quad \text{or} \quad f(n) \in O(g(n))$$

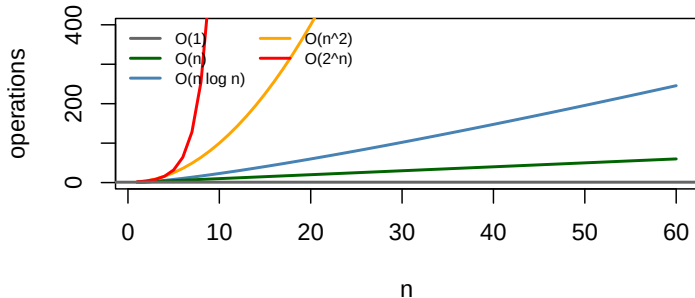
$$\exists_{C>0} \exists_{N_0 \in \mathbb{N}} \forall_{n>N_0} |f(n)| \leq C \cdot |g(n)|$$

or equivalently

$$\limsup_{n \rightarrow \infty} \frac{|f(n)|}{|g(n)|} < \infty$$

n is the problem size (e.g. the size of the input data). $f(n)$ does not, up to a scaling constant, grow faster than $g(n)$.

Big Oh: how much do the classes differ?



The constants do not matter – the shape does.

$$f(n) = n^2 + 100n + 100$$

$$f(n) = O(n^2)$$

Because $n^{-2}f(n) = 1 + 100n^{-1} + 100n^{-2} \rightarrow 1 < \infty$.

Lower order terms and constants are dropped.

Definition	Limit
$f = o(g)$ $\forall_{C>0} \exists_{N_0} \forall_{n>N_0} f \leq C g $	$\lim f / g = 0$
$f = O(g)$ $\exists_{C>0} \exists_{N_0} \forall_{n>N_0} f \leq C g $	$\limsup f / g < \infty$
$f = \omega(g)$ $\forall_{C>0} \exists_{N_0} \forall_{n>N_0} f \geq C g $	$\lim f / g = \infty$
$f = \Omega(g)$ $\exists_{C>0} \exists_{N_0} \forall_{n>N_0} f \geq Cg$	$\liminf f/g > 0$
$f = \Theta(g)$ $f = O(g) \text{ and } f = \Omega(g)$	
$f \sim g$	$\lim f/g = 1$

Complexities (the data size is a lower bound)

Big Oh	Name	Example, optimal
$O(1)$	constant	assignments, $O(1)$
$O(\log N)$	logarithmic	binary search (sorted input)
$O(N)$	linear	maximum, $O(N)$
$O(N \log N)$	log-linear	sorting, $O(N \log N)$
$O(N^2)$	quadratic	naive vector-matrix mult.
$O(N^3)$	cubic	naive matrix inversion, $O(n^{2.373})$
$O(N^3)$	cubic	naive matrix-matrix mult., $O(n^{2.373})$
$O(N^c)$	polynomial	
$O(c^N)$	exponential	brute force password cracking

Quicksort: $O(N^2)$ worst case, but $O(N \log N)$ on average.

Determine complexity: statements

```
statement 1  
statement 2  
...  
statement c
```

$$O(1)$$

Determine complexity: branches

```
if (a)
    statement a
else
    statement b
```

$$\max(O(a), O(b))$$

Determine complexity: loops

```
for (i in 1:N)  
    statement i
```

$$O(N)$$

Determine complexity: nested loops

```
for (i in 1:N)
  for (j in 1:M)
    statement i,j
```

$$O(N \cdot M)$$

Determine complexity: calling a function

```
# g(n) is O(n^2)
for (i in 1:N)
  g(i)
```

$$O(N^3)$$

Because $\sum_{i=1}^N i^2 = \Theta(N^3)$ – the cost of the body is not constant.

- naive sorting: $O(n^2)$
- **merge sort**: $O(n \log n)$, but a large number of copies – “merge sorted lists of two into four, then those and so on”
- **sort()** – **quicksort**: average (uniform) $O(n \log n)$, worst $O(n^2)$, low overhead
- **radix sort**: $O(n \cdot k)$, sorts numbers of k digits by using the digits
- **shell sort**: $O(n^{4/3})$, sorts in place by swapping elements

```
mergesort <- function(L) {  
  ## assume n = 2^k  
  n <- length(L)  
  if (n == 1) return(L)  
  L1 <- mergesort(L[1:(n / 2)])  
  L2 <- mergesort(L[(n / 2 + 1):n])  
  ## merge is done in O(n) time  
  merge(L1, L2)  
}
```

$$T(n) \leq \begin{cases} c_1 & n = 1 \\ 2T(n/2) + c_2n & n > 1 \end{cases}$$

The master theorem

A function f is **multiplicative** if $f(xy) = f(x)f(y)$.

Let $a, b, c > 0$, $k \in \mathbb{N}$ and $d(n)$ be multiplicative. Then the solution to

$$T(n) = \begin{cases} \Theta(1) & n = 1 \\ aT(n/b) + d(n) & n = b^k \end{cases}$$

is $T(n) = \Theta(a^k) + \sum_{j=0}^{k-1} a^j d(b^{k-j})$ with

$$T(n) = \begin{cases} \Theta(n^{\log_b d(b)}) & a < d(b) \\ \Theta(n^{\log_b a} \log n) & a = d(b) \\ \Theta(n^{\log_b a}) & a > d(b) \end{cases}$$

c_2n is not multiplicative, so take $T(n) = c_2\tilde{T}(n)$, then
 $\tilde{T}(1) = T(1)/c_2 = c_1/c_2 = c$ and $T(n) = 2T(n/2) + c_2n$ becomes
 $c_2\tilde{T}(n) = 2c_2\tilde{T}(n/2) + c_2n$.

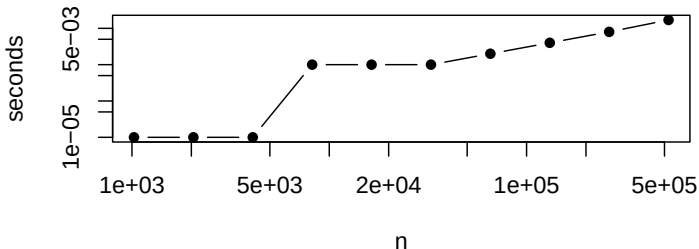
Consider

$$U(n) = \begin{cases} c & n = 1 \\ 2U(n/2) + n & n > 1 \end{cases}$$

n is multiplicative, so by the master theorem $U(n) = \Theta(n \log n)$, hence
 $U(n) \geq T(n) = O(n \log n)$. Actually $T(n) = \Theta(n \log n)$.

Measuring complexity empirically

```
ns <- 2^(10:19)
tm <- sapply(ns, \(n) system.time(sort(runif(n)))[3])
plot(ns, tm, log = "xy", type = "b", xlab = "n")
```



On a log-log scale the slope *is* the exponent.

Exercise 1

For $f(n) = (n^2 + 1)(2n^4 + 3n - 8)$, find the smallest k such that $f(n) = O(n^k)$.

$f(n) = 2n^6 + 2n^4 + 3n^3 - 8n^2 + 3n - 8$. Take $k = 6$:

$$n^{-6}f(n) = 2 + 2n^{-2} + 3n^{-3} - 8n^{-4} + 3n^{-5} - 8n^{-6} \rightarrow 2 < \infty$$

For what “simple” $g(n)$ is $f(n) = (4n \log_2 n + 1)^2 \in O(g(n))$?

$f(n) = 16n^2 \log_2^2 n + 8n \log_2 n + 1$. Take $g(n) = n^2 \log_2^2 n$:

$$\frac{f(n)}{n^2 \log_2^2 n} = 16 + 8n^{-1} \log_2^{-1} n + n^{-2} \log_2^{-2} n \rightarrow 16 < \infty$$

Exercise 3

True or false: $\log_2 n^{73} = O(\log_2 n)$?

True, since $\log_2 n^{73} = 73 \log_2 n$ and

$$\frac{73 \log_2 n}{\log_2 n} = 73 \rightarrow 73 < \infty$$

Exercise 4

Let $t_n = \sum_{k=1}^n k$. Show that $t_n \in O(n^2)$.

$$t_n = \sum_{k=1}^n k = \frac{n(n+1)}{2} = \frac{n^2}{2} + \frac{n}{2}$$

and then

$$n^{-2}t_n = \frac{1}{2} + \frac{1}{2}n^{-1} \rightarrow \frac{1}{2} < \infty$$

More exercises in

732A94_AdvancedRHT2026_Lecture06_P1_ComputationalComplexityExamples

If we cannot solve a hard problem, let us approximate its solution.

Let S_{opt} be the optimal solution and S_{approx} the approximate one.

k -absolute approximate algorithm if $|S_{opt} - S_{approx}| \leq k$

k -(relative) approximate algorithm if $s \leq k$, where

$$s = \max(S_{opt}/S_{approx}, S_{approx} - S_{opt})$$

LAB: the knapsack problem.

Vertex covering problem: for a graph $G = (V, E)$ find the smallest subset $V' \subseteq V$ such that each edge in E has an ending in V' . This is NP-hard.

Maximal matching problem: find a set M of edges in E with no common vertices, that is maximal (no edge can be added to it).

A maximal matching can be found greedily in polynomial time:

1. $M := \emptyset$
2. Choose any edge e , set $M := M \cup \{e\}$
3. Remove the ends of e from the graph
4. Repeat until the graph is empty

Example: a 2-approximation

Any maximal matching induces a vertex cover – otherwise there would be an edge without an ending in the induced vertex set, and that edge could have been added to the matching.

Finding a maximal matching is a 2-approximate algorithm for the vertex covering problem.

Proof. If $OPT = |V'|$ then $|M| \leq OPT$, since any vertex cover must contain at least one ending of each matching edge. The cover produced by the approximation has size $2|M| \leq 2OPT$, so $S_{approx} \leq 2S_{opt}$. $\square$

Vazirani, V. V. (2003) *Approximation algorithms*, Springer.

Questions?

See you next time!